

TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos



Profesores:

Ing. Roly Steeven Cedeño Menéndez, Mtr.

Titulaciones	Semestre
• <i>Tecnologías de la Información en Línea</i>	<i>Quinto</i>

NOMBRES Y APELLIDOS: JHONY ALBERTO AGUILAR RODRIGUEZ

PARALELO: A

SEMESTRE: QUINTO

ACTIVIDAD: Actividad #8: Desarrollo de sistemas y bases de datos

PERIODO SEPTIEMBRE 2025 - ENERO 2026



Actividades a desarrollar en la Unidad 4

Actividad # 8

En esta actividad usted deberá construir la arquitectura del servidor, configurar la base de datos y exponer los servicios mediante una API REST.

Objetivo de la tarea:

- Construir la arquitectura del lado del servidor (Backend) y la base de datos del Sistema de Gestión de Incidentes, aplicando tecnologías de nivel industrial (Node.js o Java Spring Boot) para exponer una API RESTful escalable y segura.

Instrucciones para el Desarrollo

- Diseñe y configure la base de datos (MongoDB o base relacional) para la gestión de tickets.
- Desarrolle los endpoints RESTful para permitir las operaciones CRUD completas (GET, POST, PUT, DELETE).
- Realice las pruebas de conectividad de sus rutas utilizando herramientas como Postman o cURL.
- Suba el código del servidor a una rama feature/backend-api en su repositorio de GitHub.



DESARROLLO

CONFIGURACIÓN DE LA BASE DE DATOS

- Crear una base de datos documental (MongoDB) o relacional (H2/PostgreSQL) según el stack elegido.

CODIGOS:

```
CREATE DATABASE helpdesk;
```

```
\c helpdesk
```

CREACION DE LA TABLA

```
CREATE TABLE tickets (  
  id SERIAL PRIMARY KEY,  
  titulo VARCHAR(150) NOT NULL,  
  descripcion TEXT NOT NULL,  
  categoria VARCHAR(20) NOT NULL  
    CHECK (categoria IN ('Red', 'Hardware', 'Software')),  
  prioridad VARCHAR(10) NOT NULL  
    CHECK (prioridad IN ('Alta', 'Media', 'Baja')),  
  estado VARCHAR(20) NOT NULL  
    DEFAULT 'Abierto'  
    CHECK (estado IN ('Abierto', 'En Progreso', 'Cerrado'))  
);
```

Tickets Registrados

Tabla de Tickets

ID	Incidente	Categoría	Prioridad	Estado	Acciones
2	Prueba de conexión	Hardware	Media	Abierto	Ver detalle Cambiar estado Eliminar
3	Falla en Aplicación	Software	Baja	Abierto	Ver detalle Cambiar estado Eliminar



- Diseñar la colección/tabla Tickets con los atributos mínimos: id, título, descripción, categoría (Red, Hardware, Software), prioridad (Alta, Media, Baja) y estado (Abierto, En Progreso, Cerrado).

CODIGO:

```
CREATE TABLE tickets (  
  id SERIAL PRIMARY KEY,  
  titulo VARCHAR(150) NOT NULL,  
  descripcion TEXT NOT NULL,  
  categoria VARCHAR(20) NOT NULL  
    CHECK (categoria IN ('Red', 'Hardware', 'Software')),  
  prioridad VARCHAR(10) NOT NULL  
    CHECK (prioridad IN ('Alta', 'Media', 'Baja')),  
  estado VARCHAR(20) NOT NULL  
    DEFAULT 'Abierto'  
    CHECK (estado IN ('Abierto', 'En Progreso', 'Cerrado'))  
);
```

Resumen del Sistema

Tickets Registrados

Total de tickets: 2

Tickets Abiertos

Total de tickets abiertos: 2

Tickets Cerrados

Total de tickets cerrados: 0



DESARROLLO DE LA API REST

- Desarrollar los endpoints de comunicación HTTP:
 - o **GET /tickets**: Para listar todos los incidentes.

CODIGO:

```
app.get('/tickets', async (req, res) => {  
  try {  
    const resultado = await pool.query(  
      'SELECT * FROM tickets ORDER BY id'  
    );  
    res.json(resultado.rows);  
  } catch (error) {  
    console.error('Error al obtener tickets:', error);  
    res.status(500).json({  
      mensaje: 'Error al obtener los tickets',  
      error: error.message  
    });  
  }  
});
```

Invoke-RestMethod -Uri "http://localhost:3000/tickets" -Method GET

```
PS E:\jhony\UTH\JHONY\QUINTO SEMESTRE\DESARROLLO DE SISTEMAS INFORMATICOS\helpdesk-html\backend> node server.js  
Servidor ejecutándose en http://localhost:3000  
PS E:\jhony\UTH\JHONY\QUINTO SEMESTRE\DESARROLLO DE SISTEMAS INFORMATICOS\helpdesk-html\backend> Invoke-RestMethod -Uri "http://localhost:3000/tickets" -Method GET  
  
id      : 2  
titulo  : Prueba de conexión  
descripcion : Prueba de conexión entre el front y el backend  
categoria : Hardware  
prioridad : Media  
estado  : Abierto  
  
id      : 3  
titulo  : Falla en Aplicación  
descripcion : La aplicación presenta un error al momento de iniciar.  
categoria : Software  
prioridad : Baja  
estado  : Abierto
```



Sistema Help Desk

Listado de Tickets Registrados

[Inicio](#) [Reportar Incidente](#) [Ver Tickets](#)

Tickets Registrados

Tabla de Tickets

ID	Incidente	Categoría	Prioridad	Estado	Acciones
2	Prueba API REST	Software	Media	En Progreso	Ver detalle Cambiar estado Eliminar
3	Falla en Aplicación	Software	Baja	Abierto	Ver detalle Cambiar estado Eliminar
4	Prueba API REST	Software	Baja	Abierto	Ver detalle Cambiar estado Eliminar



- **GET /tickets/{id}**: Para buscar un ticket específico.

CODIGO:

```
app.get('/tickets/:id', async (req, res) => {
  try {
    const resultado = await pool.query(
      'SELECT * FROM tickets WHERE id = $1',
      [req.params.id]
    );
    if (resultado.rows.length === 0) {
      return res.status(404).json({
        mensaje: 'Ticket no encontrado'
      });
    }
    res.json(resultado.rows[0]);
  } catch (error) {
    console.error('Error al obtener ticket:', error);
    res.status(500).json({
      mensaje: 'Error al obtener el ticket',
      error: error.message
    });
  }
});
```

Invoke-RestMethod -Uri "http://localhost:3000/tickets/2" -Method GET

```
PS E:\Jhony\UTM\JHONY\QUINTO SEMESTRE\DESARROLLO DE SISTEMAS INFORMATICOS\helpdesk-html\backend> Invoke-RestMethod -Uri "http://localhost:3000/tickets/2" -Method GET

id      : 2
titulo   : Prueba de conexión
descripcion : Prueba de conexión entre el front y el backend
categoria : Hardware
prioridad : Media
estado   : Abierto
```

The screenshot shows a web application interface for managing tickets. At the top, there's a navigation bar with 'Aplicaciones' and a hamburger menu icon. Below it, a blue header says 'Listado de Tickets Registrados'. A dark blue bar contains links: 'Inicio', 'Reportar Incidente', and 'Ver Tickets'. The main content area is titled 'Tickets Registrados' and features a table labeled 'Tabla de Tickets'. The table has columns: ID, Incidente, Categoría, Prioridad, Estado, and Acciones. One ticket is listed with ID 2, titled 'Prueba API REST', category 'Software', priority 'Media' (highlighted in a brown circle), and status 'En Progreso'. The 'Acciones' column for this ticket contains three buttons: 'Ver detalle', 'Cambiar estado', and 'Eliminar'. On the right side, a modal window is open, titled '127.0.0.1:5500 dice' and 'DETALLE DEL TICKET'. It displays the details of the selected ticket: ID: 2, Título: Prueba API REST, Categoría: Software, Prioridad: Media, Estado: En Progreso, and Descripción: Ticket actualizado correctamente. An 'Aceptar' button is at the bottom right of the modal.

ID	Incidente	Categoría	Prioridad	Estado	Acciones
2	Prueba API REST	Software	Media	En Progreso	Ver detalle Cambiar estado Eliminar



- **POST /tickets**: Para registrar un nuevo incidente.

CODIGO:

```
app.post('/tickets', async (req, res) => {
  try {
    const {
      titulo,
      descripcion,
      categoria,
      prioridad,
      estado
    } = req.body;
    const resultado = await pool.query(
      `INSERT INTO tickets
      (titulo, descripcion, categoria, prioridad, estado)
      VALUES ($1, $2, $3, $4, $5)
      RETURNING *`,
      [
        titulo,
        descripcion,
        categoria,
        prioridad,
        estado || 'Abierto'
      ]
    );
    res.status(201).json(resultado.rows[0]);

  } catch (error) {
    console.error('Error al crear ticket:', error);
    res.status(500).json({
      mensaje: 'Error al crear el ticket',
      error: error.message
    });
  }
});
```

Invoke-RestMethod -Uri "http://localhost:3000/tickets" -Method POST -ContentType "application/json" -Body '{"titulo":"Prueba API REST","descripcion":"Prueba del endpoint POST para registrar un incidente.", "categoria":"Software", "prioridad":"Baja", "estado":"Abierto"}'



```
PS E:\Jhony\UTM\JHONY\QUINTO SEMESTRE\DESARROLLO DE SISTEMAS INFORMATICOS\helpdesk-html\backend> Invoke-RestMethod -Uri "http://localhost:3000/tickets" -Method POST -ContentType "application/json" -Body '{"titulo":"Prueba API REST","descripcion":"Prueba del endpoint POST para registrar un incidente.", "categoria":"Software", "prioridad":"Baja", "estado":"Abierto"}'
```

```
id          : 4
titulo      : Prueba API REST
descripcion : Prueba del endpoint POST para registrar un incidente.
categoria   : Software
prioridad   : Baja
estado      : Abierto
```

127.0.0.1:5500 dice

¡Ticket registrado correctamente!

Aceptar

- **PUT /tickets/{id}**: Para actualizar el estado o detalles de un ticket.

CODIGO:

```
app.put('/tickets/:id', async (req, res) => {
  try {
    const {
      titulo,
      descripcion,
      categoria,
      prioridad,
      estado
    } = req.body;
    const resultado = await pool.query(
      `UPDATE tickets
      SET titulo = $1,
        descripcion = $2,
        categoria = $3,
        prioridad = $4,
        estado = $5
      WHERE id = $6
      RETURNING *`,
      [
        titulo,
        descripcion,
        categoria,
        prioridad,
        estado,
        req.params.id
      ]
    )
  }
```



```
);  
if (resultado.rows.length === 0) {  
    return res.status(404).json({  
        mensaje: 'Ticket no encontrado'  
    });  
}  
res.json(resultado.rows[0]);  
} catch (error) {  
    console.error('Error al actualizar ticket:', error);  
  
    res.status(500).json({  
        mensaje: 'Error al actualizar el ticket',  
        error: error.message  
    });  
}  
});
```

Invoke-RestMethod -Uri "http://localhost:3000/tickets/2" -Method PUT -ContentType "application/json" -Body '{"titulo":"Prueba API REST","descripcion":"Ticket actualizado correctamente.", "categoria":"Software", "prioridad":"Media", "estado":"En Progreso"}'

```
Invoke-RestMethod -Uri "http://localhost:3000/tickets/2" -Method PUT -ContentType "application/json" -Body '{"titulo":"Prueba API REST","descripcion":"Ticket actualizado correctamente.", "categoria":"Software", "prioridad":"Media", "estado":"En Progreso"}' ISTEMAS INFORMATICOS\helpdesk-html\backend>
```

```
id      : 2  
titulo  : Prueba API REST  
descripcion : Ticket actualizado correctamente.  
categoria : Software  
prioridad : Media  
estado  : En Progreso
```

es

Tabla de Tickets

ID	Incidente	Categoría	Prioridad	Estado
2	Prueba API REST	Software	Media	En Progreso

127.0.0.1:5500 dice

DETALLE DEL TICKET

ID: 2
Título: Prueba API REST
Categoría: Software
Prioridad: Media
Estado: En Progreso

Descripción:
Ticket actualizado correctamente.

Aceptar

Eliminar

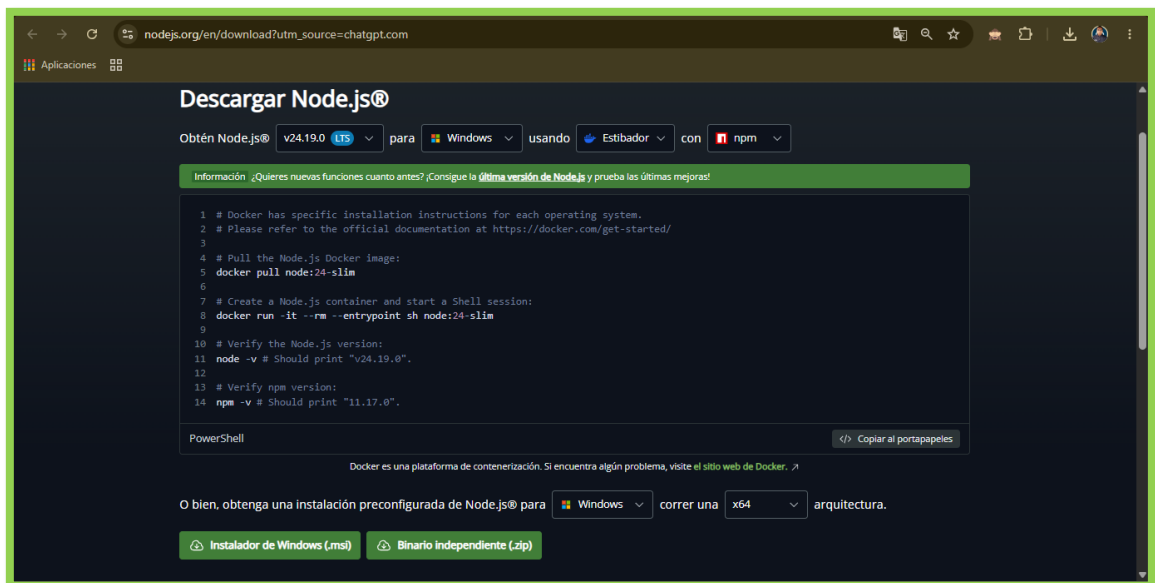
- **DELETE /tickets/{id}**: Para eliminar un registro.

CODIGO:

```
app.delete('/tickets/:id', async (req, res) => {
  try {
    const resultado = await pool.query(
      'DELETE FROM tickets WHERE id = $1 RETURNING *',
      [req.params.id]
    );
    if (resultado.rows.length === 0) {
      return res.status(404).json({
        mensaje: 'Ticket no encontrado'
      });
    }
    res.json({
      mensaje: 'Ticket eliminado correctamente',
      ticket: resultado.rows[0]
    });
  } catch (error) {
    console.error('Error al eliminar ticket:', error);
    res.status(500).json({
      mensaje: 'Error al eliminar el ticket',
      error: error.message
    });
  }
});
```

Invoke-RestMethod -Uri "http://localhost:3000/tickets/4" -Method DELETE

INSTALACION DEL NODE



The screenshot shows the Node.js download page. At the top, it says 'Descargar Node.js®'. Below this, there are dropdown menus to select the version (v24.19.0 LTS), platform (Windows), architecture (x64), and package manager (npm). A green banner contains information about the latest version. Below the banner, there is a code block with instructions for installing Node.js using Docker. At the bottom, there are buttons for 'Instalador de Windows (.msi)' and 'Binario independiente (.zip)'.

```

Administrador: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\Windows\system32> & "C:\Program Files\PostgreSQL\18\bin\pg_isready.exe"
:5432 - aceptando conexiones
PS C:\Windows\system32> psql -U postgres -h localhost -p 5432
psql : El término 'psql' no se reconoce como nombre de un cmdlet, función, archivo de script o programa ejecutable.
Compruebe si escribió correctamente el nombre o, si incluyó una ruta de acceso, compruebe que dicha ruta es correcta e
inténtelo de nuevo.
En línea: 1 Carácter: 1
+ psql -U postgres -h localhost -p 5432
~
+ CategoryInfo          : ObjectNotFound: (psql:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException

PS C:\Windows\system32> & "C:\Program Files\PostgreSQL\18\bin\psql.exe" -U postgres -h localhost -p 5432
Contraseña para usuario postgres:
psql (18.4)
ADVERTENCIA: El código de página de la consola (850) difiere del código
de página de Windows (1252).
Los caracteres de 8 bits pueden funcionar incorrectamente.
Vea la página de referencia de psql «Notes for Windows users»
para obtener más detalles.
Dígame «help» para obtener ayuda.

postgres=# CREATE DATABASE helpdesk;
CREATE DATABASE
postgres=# \c helpdesk
Ahora está conectado a la base de datos «helpdesk» con el usuario «postgres».
helpdesk=# CREATE TABLE tickets (
helpdesk(#         id SERIAL PRIMARY KEY,
helpdesk(#         titulo VARCHAR(150) NOT NULL,
helpdesk(#         descripcion TEXT NOT NULL,
helpdesk(#         categoria VARCHAR(20) NOT NULL CHECK (categoria IN ('Red', 'Hardware', 'Software')),
helpdesk(#         prioridad VARCHAR(10) NOT NULL CHECK (prioridad IN ('Alta', 'Media', 'Baja')),
helpdesk(#         estado VARCHAR(20) NOT NULL DEFAULT 'Abierto' CHECK (estado IN ('Abierto', 'En Progreso', 'Cerrado')))
helpdesk(# );
CREATE TABLE
helpdesk=# \d tickets

```

Columna	Tipo	Tabla %public.tickets%	Ordenamiento	Nullable	Por omisi‑n
id	integer			not null	nextval('tickets_id_seq'::regclass)
titulo	character varying(150)			not null	
descripcion	text			not null	
categoria	character varying(20)			not null	
prioridad	character varying(10)			not null	
estado	character varying(20)			not null	'Abierto'::character varying

```

ndices:
"tickets_pkey" PRIMARY KEY, btree (id)
Restricciones CHECK:
"tickets_categoria_check" CHECK (categoria::text = ANY (ARRAY['Red'::character varying, 'Hardware'::character varying, 'Software'::character varying]::text[]))
"tickets_estado_check" CHECK (estado::text = ANY (ARRAY['Abierto'::character varying, 'En Progreso'::character varying, 'Cerrado'::character varying]::text[]))
"tickets_prioridad_check" CHECK (prioridad::text = ANY (ARRAY['Alta'::character varying, 'Media'::character varying, 'Baja'::character varying]::text[]))

helpdesk=# SELECT * FROM tickets;
 id | descripcion | categoria | prioridad | estado
-----+-----+-----+-----+-----
(0 filas)

```

[illegible]



TERMINAL 1

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS E:\Jhony\UTM\JHONY\QUINTO SEMESTRE\DESARROLLO DE SISTEMAS INFORMATICOS\helpdesk-html> & "C:\Program Files\PostgreSQL\18\bin\psql.exe" --version
psql (PostgreSQL) 18.4
PS E:\Jhony\UTM\JHONY\QUINTO SEMESTRE\DESARROLLO DE SISTEMAS INFORMATICOS\helpdesk-html> cd backend
PS E:\Jhony\UTM\JHONY\QUINTO SEMESTRE\DESARROLLO DE SISTEMAS INFORMATICOS\helpdesk-html\backend> node server.js
Servidor ejecutándose en http://localhost:3000
PS E:\Jhony\UTM\JHONY\QUINTO SEMESTRE\DESARROLLO DE SISTEMAS INFORMATICOS\helpdesk-html\backend> node server.js
Servidor ejecutándose en http://localhost:3000
PS E:\Jhony\UTM\JHONY\QUINTO SEMESTRE\DESARROLLO DE SISTEMAS INFORMATICOS\helpdesk-html\backend> node server.js
Servidor ejecutándose en http://localhost:3000
PS E:\Jhony\UTM\JHONY\QUINTO SEMESTRE\DESARROLLO DE SISTEMAS INFORMATICOS\helpdesk-html\backend> node server.js
Servidor ejecutándose en http://localhost:3000
PS E:\Jhony\UTM\JHONY\QUINTO SEMESTRE\DESARROLLO DE SISTEMAS INFORMATICOS\helpdesk-html\backend> node server.js
Servidor ejecutándose en http://localhost:3000
PS E:\Jhony\UTM\JHONY\QUINTO SEMESTRE\DESARROLLO DE SISTEMAS INFORMATICOS\helpdesk-html\backend> node server.js
Servidor ejecutándose en http://localhost:3000
PS E:\Jhony\UTM\JHONY\QUINTO SEMESTRE\DESARROLLO DE SISTEMAS INFORMATICOS\helpdesk-html\backend> node server.js
Servidor ejecutándose en http://localhost:3000
PS E:\Jhony\UTM\JHONY\QUINTO SEMESTRE\DESARROLLO DE SISTEMAS INFORMATICOS\helpdesk-html\backend> node server.js
Servidor ejecutándose en http://localhost:3000
```

TERMINAL 2

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS E:\Jhony\UTM\JHONY\QUINTO SEMESTRE\DESARROLLO DE SISTEMAS INFORMATICOS\helpdesk-html> Invoke-RestMethod -Uri "http://localhost:3000/tickets/1" -Method GET
id      : 1
titulo  : Computadora no enciende
descripcion : El equipo no inicia correctamente al presionar el botón de encendido.
categoria : Hardware
prioridad : Alta
estado  : Abierto

PS E:\Jhony\UTM\JHONY\QUINTO SEMESTRE\DESARROLLO DE SISTEMAS INFORMATICOS\helpdesk-html> Invoke-RestMethod -Uri "http://localhost:3000/tickets/1" -Method PUT -ContentType "application/json" -Body '{"titulo":"Computadora no enciende","descripcion":"El equipo presenta falla al encender y requiere revisión técnica.", "categoria":"Hardware","prioridad":"Media","estado":"En Progreso"}'
id      : 1
titulo  : Computadora no enciende
descripcion : El equipo presenta falla al encender y requiere revisión técnica.
categoria : Hardware
prioridad : Media
estado  : En Progreso

PS E:\Jhony\UTM\JHONY\QUINTO SEMESTRE\DESARROLLO DE SISTEMAS INFORMATICOS\helpdesk-html> Invoke-RestMethod -Uri "http://localhost:3000/tickets/1" -Method DELETE
mensaje      ticket
-----
Ticket eliminado correctamente @({id=1; titulo=Computadora no enciende; descripcion=El equipo presenta falla al encender y requiere revisión técnica.; categoria=Hardware; prioridad=Media; estado=En Progreso})

PS E:\Jhony\UTM\JHONY\QUINTO SEMESTRE\DESARROLLO DE SISTEMAS INFORMATICOS\helpdesk-html> Invoke-RestMethod -Uri "http://localhost:3000/tickets" -Method GET
PS E:\Jhony\UTM\JHONY\QUINTO SEMESTRE\DESARROLLO DE SISTEMAS INFORMATICOS\helpdesk-html> cd backend
PS E:\Jhony\UTM\JHONY\QUINTO SEMESTRE\DESARROLLO DE SISTEMAS INFORMATICOS\helpdesk-html\backend> npm install cors
added 2 packages, and audited 84 packages in 3s
27 packages are looking for funding
run 'npm fund' for details
```

Bibliografía

Diseño Estructurado de Sistemas. (2021). Retrieved 02 Mayo 2021, from <https://users.exa.unicen.edu.ar/catedras/prog1/sites/default/files/ApuntesDiagramaEstructura.Pdf>

Object Oriented Design. (2021). Retrieved 02 Mayo 2021, from <https://www.professionalqa.com/object-oriented-design>

Diseño a nivel de componentes. (2021). Retrieved 04 Mayo 2021, from <https://virtual.itca.edu.sv/Mediadores/stis/37 diseo a nivel de componentes.html>